

El protocolo HTTP

José Manuel Sánchez Álvarez

16 de septiembre de 2025

Índice de contenidos

Introducción al Protocolo HTTP	1
La Comunicación entre el Navegador y el Servidor mediante el Protocolo HTTP	2
Solicitud HTTP	2
Lista de Comandos HTTP (Métodos HTTP)	2
Encabezados	3
Respuesta HTTP	3
Encabezados de Respuesta	4
Los Códigos de Respuesta	4
1xx: Mensajes de Información	4
2xx: Éxito	4
3xx: Redirección	5
4xx: Errores del Cliente	5
5xx: Errores del Servidor	5
Códigos Relevantes en APIs RESTful	6
Tipos MIME	6
1. Tipos de Texto	6
2. Tipos de Imágenes	7
3. Tipos de Aplicaciones	7
4. Tipos de Audio	8
5. Tipos de Video	8
6. Tipos de Fuentes	8
7. Tipos de Datos Relacionados con Modelos 3D	9
8. Tipos Personalizados	9
Tipos MIME Más Usados en Desarrollo Web y APIs RESTful	9
Conclusión	10

Introducción al Protocolo HTTP

El **Protocolo de Transferencia de Hipertexto (HTTP)** es el fundamento de la comunicación en la web. Es un protocolo de aplicación basado en texto que permite la transferencia de recursos entre un cliente (generalmente un navegador web) y un servidor. HTTP opera sobre el modelo **cliente-servidor**, donde el cliente envía una solicitud y el servidor responde con los datos solicitados o un mensaje de estado.

HTTP es **sin estado**, lo que significa que cada solicitud es independiente y no guarda información entre solicitudes. Sin embargo, se pueden usar tecnologías como **cookies** y **sesiones** para mantener el estado entre interacciones.

La Comunicación entre el Navegador y el Servidor mediante el Protocolo HTTP

La comunicación entre el navegador (cliente) y el servidor sigue un flujo básico:

1. **Solicitud del Cliente:**
 - El navegador envía una solicitud HTTP al servidor indicando qué recurso necesita.
 - Ejemplo: Solicitar la página principal de un sitio web.
2. **Procesamiento en el Servidor:**
 - El servidor recibe la solicitud, procesa la lógica necesaria (consulta a bases de datos, generación de contenido dinámico, etc.) y prepara una respuesta.
3. **Respuesta del Servidor:**
 - El servidor envía una respuesta HTTP al navegador con el recurso solicitado o un mensaje de estado.
 - Ejemplo: Devolver el código HTML de la página principal.
4. **Renderizado en el Navegador:**
 - El navegador interpreta la respuesta y muestra el contenido al usuario.

Este proceso puede repetirse varias veces durante una sesión, por ejemplo, cuando se cargan imágenes, archivos CSS o JavaScript adicionales.

Solicitud HTTP

Una **solicitud HTTP** es el mensaje que el cliente envía al servidor para pedir un recurso o realizar una acción. Está compuesta por tres partes principales:

1. **Línea de solicitud:**
 - Contiene el método HTTP, la URL del recurso y la versión del protocolo.
 - Ejemplo:
`GET /index.html HTTP/1.1`
2. **Encabezados:**
 - Proporcionan metadatos sobre la solicitud.
 - Ejemplo:
`Host: www.ejemplo.com`
`User-Agent: Mozilla/5.0`
`Accept: text/html`
3. **Cuerpo (opcional):**
 - Contiene datos enviados al servidor, generalmente en solicitudes POST o PUT.
 - Ejemplo:
`{`
 `"nombre": "Juan",`
 `"edad": 25`
`}`

Lista de Comandos HTTP (Métodos HTTP)

Los métodos HTTP definen la acción que se realizará sobre un recurso. Los más comunes son:

1. **GET:**
 - Solicita un recurso específico.
 - No debe tener efectos secundarios en el servidor.
2. **POST:**
 - Envía datos al servidor para crear un nuevo recurso.
 - Ejemplo: Enviar un formulario.

3. **PUT:**
 - Actualiza un recurso existente o crea uno nuevo si no existe.
 4. **DELETE:**
 - Elimina un recurso específico.
 5. **PATCH:**
 - Realiza una actualización parcial de un recurso.
 6. **HEAD:**
 - Similar a GET, pero solo solicita los encabezados sin el cuerpo de la respuesta.
 7. **OPTIONS:**
 - Solicita información sobre los métodos HTTP permitidos para un recurso.
-

Encabezados

Los encabezados HTTP proporcionan información adicional sobre la solicitud o respuesta. Se dividen en categorías:

1. **Encabezados de solicitud:**
 - **Host:** Indica el dominio del servidor.
 - **User-Agent:** Informa sobre el navegador o cliente.
 - **Accept:** Especifica los tipos de contenido que el cliente puede manejar.
 2. **Encabezados de respuesta:**
 - **Content-Type:** Indica el tipo de contenido devuelto (p. ej., `text/html`).
 - **Cache-Control:** Define políticas de caché.
 - **Set-Cookie:** Envía cookies al cliente.
-

Respuesta HTTP

Una **respuesta HTTP** es el mensaje que el servidor envía al cliente después de procesar la solicitud. Está compuesta por tres partes principales:

1. **Línea de estado:**
 - Contiene el código de estado HTTP, la versión del protocolo y un mensaje descriptivo.
 - Ejemplo:
`HTTP/1.1 200 OK`
2. **Encabezados:**
 - Proporcionan metadatos sobre la respuesta.
 - Ejemplo:
`Content-Type: text/html`
`Content-Length: 1234`
3. **Cuerpo (opcional):**
 - Contiene los datos del recurso solicitado.
 - Ejemplo:

```
<html>
  <body>
    <h1>Bienvenido</h1>
  </body>
</html>
```

Encabezados de Respuesta

Algunos encabezados comunes en las respuestas HTTP incluyen:

1. **Content-Type:**
 - Indica el tipo de contenido devuelto (p. ej., `application/json`, `text/html`).
 2. **Content-Length:**
 - Especifica el tamaño del cuerpo de la respuesta en bytes.
 3. **Location:**
 - Redirige al cliente a una nueva URL.
 4. **Set-Cookie:**
 - Envía cookies al cliente para mantener el estado.
-

Los Códigos de Respuesta

Los **códigos de estado HTTP** son números de tres dígitos que indican el resultado de una solicitud HTTP. Se dividen en cinco categorías principales, y cada categoría tiene un propósito específico. A continuación, se detalla cada categoría con ejemplos relevantes, especialmente enfocados en el uso en APIs RESTful.

1xx: Mensajes de Información

Estos códigos indican que la solicitud ha sido recibida y está en proceso. No se utilizaban en las primeras versiones del protocolo HTTP (1.0), pero son útiles en HTTP/1.1 y versiones posteriores.

- **100 Continue:**
 - Indica que el servidor ha recibido la solicitud inicial y el cliente puede continuar enviando el resto de los datos.
 - Usado comúnmente en solicitudes grandes para confirmar que el servidor está listo para procesarlas.
 - **101 Switching Protocols:**
 - El servidor acepta cambiar el protocolo de comunicación (por ejemplo, de HTTP a WebSocket).
 - Este código es útil en aplicaciones en tiempo real.
-

2xx: Éxito

Estos códigos indican que la solicitud fue exitosa y se procesó correctamente.

- **200 OK:**
 - La solicitud se completó correctamente.
 - En APIs RESTful, este código se usa para respuestas exitosas en operaciones GET, PUT o PATCH.
- **201 Created:**
 - Indica que la solicitud POST fue exitosa y se creó un nuevo recurso.
 - La respuesta debe incluir la URL del recurso creado en el encabezado `Location`.
- **202 Accepted:**
 - La solicitud ha sido aceptada, pero el procesamiento aún no se ha completado.
 - Útil en operaciones asíncronas, como tareas en segundo plano.
- **204 No Content:**
 - La solicitud fue exitosa, pero no hay contenido para devolver.
 - Usado comúnmente en operaciones DELETE o PUT donde no se necesita una respuesta.
- **206 Partial Content:**
 - Indica que solo se devuelve parte del recurso solicitado.

- Usado en APIs RESTful cuando se solicitan rangos específicos de datos (p. ej., paginación).
-

3xx: Redirección

Estos códigos indican que el recurso solicitado ha cambiado de ubicación o que el cliente debe realizar acciones adicionales.

- **301 Moved Permanently:**
 - El recurso solicitado ha sido movido permanentemente a una nueva URL.
 - Los clientes deben actualizar sus enlaces.
 - **302 Found:**
 - El recurso solicitado se encuentra temporalmente en una nueva URL.
 - Difiere de 301 porque no implica un cambio permanente.
 - **304 Not Modified:**
 - Indica que el recurso no ha cambiado desde la última solicitud.
 - Usado para mejorar el rendimiento mediante el uso de caché.
 - **307 Temporary Redirect:**
 - Similar a 302, pero especifica que el método HTTP no debe cambiar durante la redirección.
 - **308 Permanent Redirect:**
 - Similar a 301, pero garantiza que el método HTTP no cambie.
-

4xx: Errores del Cliente

Estos códigos indican que el cliente cometió un error al realizar la solicitud.

- **400 Bad Request:**
 - La solicitud no se pudo entender debido a una sintaxis incorrecta.
 - Usado en APIs RESTful cuando los datos enviados por el cliente son inválidos.
 - **401 Unauthorized:**
 - El cliente no está autenticado.
 - Usado comúnmente en APIs RESTful para solicitar credenciales o tokens JWT.
 - **403 Forbidden:**
 - El cliente está autenticado, pero no tiene permiso para acceder al recurso.
 - Usado para implementar control de acceso.
 - **404 Not Found:**
 - El recurso solicitado no existe en el servidor.
 - Usado en APIs RESTful cuando el recurso no se encuentra.
 - **405 Method Not Allowed:**
 - El método HTTP utilizado no está permitido para el recurso.
 - Ejemplo: Usar POST en un endpoint que solo admite GET.
 - **409 Conflict:**
 - Indica un conflicto en la solicitud, como intentar crear un recurso que ya existe.
 - Usado en APIs RESTful para manejar conflictos de datos.
 - **422 Unprocessable Entity:**
 - La solicitud es válida, pero contiene errores semánticos.
 - Usado en APIs RESTful para validaciones complejas.
-

5xx: Errores del Servidor

Estos códigos indican que el servidor falló al procesar la solicitud.

- **500 Internal Server Error:**
 - Error genérico del servidor.
 - Usado cuando ocurre un problema inesperado.
 - **501 Not Implemented:**
 - El servidor no soporta la funcionalidad requerida.
 - Usado cuando un método HTTP no está implementado.
 - **502 Bad Gateway:**
 - El servidor actuaba como puerta de enlace y recibió una respuesta inválida del servidor upstream.
 - **503 Service Unavailable:**
 - El servidor no está disponible temporalmente (p. ej., sobrecarga o mantenimiento).
 - **504 Gateway Timeout:**
 - El servidor no recibió una respuesta a tiempo del servidor upstream.
-

Códigos Relevantes en APIs RESTful

En el desarrollo de APIs RESTful, algunos códigos son especialmente útiles:

1. **GET:**
 - **200 OK:** Respuesta exitosa.
 - **404 Not Found:** El recurso no existe.
 2. **POST:**
 - **201 Created:** Recurso creado exitosamente.
 - **400 Bad Request:** Datos inválidos.
 3. **PUT/PATCH:**
 - **200 OK:** Actualización exitosa.
 - **204 No Content:** Actualización sin respuesta.
 - **409 Conflict:** Conflicto en la actualización.
 4. **DELETE:**
 - **204 No Content:** Recurso eliminado.
 - **404 Not Found:** El recurso no existe.
-

Tipos MIME

Los **tipos MIME (Multipurpose Internet Mail Extensions)** son estándares que indican el tipo de contenido que se está transfiriendo entre el cliente y el servidor. Se utilizan principalmente en el encabezado **Content-Type** de las respuestas HTTP para informar al cliente sobre el formato de los datos enviados. A continuación, se presenta una lista ampliada de tipos MIME, organizada por categorías.

1. Tipos de Texto

Estos tipos representan contenido basado en texto.

- **text/plain:**
 - Contenido de texto sin formato.
 - Ejemplo: Archivos `.txt`.
- **text/html:**
 - Contenido HTML.
 - Ejemplo: Páginas web.
- **text/css:**

- Hojas de estilo CSS.
 - Ejemplo: Archivos `.css`.
 - **text/javascript:**
 - Código JavaScript.
 - Ejemplo: Archivos `.js`.
 - **text/csv:**
 - Datos separados por comas (CSV).
 - Ejemplo: Archivos `.csv`.
 - **text/xml:**
 - Datos en formato XML.
 - Ejemplo: Archivos `.xml`.
-

2. Tipos de Imágenes

Estos tipos representan formatos de imágenes.

- **image/jpeg:**
 - Imágenes en formato JPEG.
 - Ejemplo: Archivos `.jpg`, `.jpeg`.
 - **image/png:**
 - Imágenes en formato PNG.
 - Ejemplo: Archivos `.png`.
 - **image/gif:**
 - Imágenes animadas o estáticas en formato GIF.
 - Ejemplo: Archivos `.gif`.
 - **image/svg+xml:**
 - Imágenes vectoriales en formato SVG.
 - Ejemplo: Archivos `.svg`.
 - **image/webp:**
 - Imágenes en formato WebP.
 - Ejemplo: Archivos `.webp`.
 - **image/bmp:**
 - Imágenes en formato BMP.
 - Ejemplo: Archivos `.bmp`.
-

3. Tipos de Aplicaciones

Estos tipos representan datos estructurados o binarios.

- **application/json:**
 - Datos en formato JSON.
 - Ejemplo: Respuestas de APIs RESTful.
- **application/xml:**
 - Datos en formato XML.
 - Ejemplo: Archivos `.xml`.
- **application/pdf:**
 - Documentos PDF.
 - Ejemplo: Archivos `.pdf`.
- **application/zip:**
 - Archivos comprimidos en formato ZIP.
 - Ejemplo: Archivos `.zip`.

- **application/octet-stream:**
 - Datos binarios genéricos.
 - Ejemplo: Descargas de archivos ejecutables.
 - **application/x-www-form-urlencoded:**
 - Datos codificados en formularios HTML.
 - Ejemplo: Envío de formularios POST.
 - **application/javascript:**
 - Código JavaScript.
 - Ejemplo: Archivos `.js`.
 - **application/vnd.api+json:**
 - Datos JSON específicos de APIs siguiendo el estándar JSON:API.
-

4. Tipos de Audio

Estos tipos representan formatos de audio.

- **audio/mpeg:**
 - Archivos de audio en formato MP3.
 - Ejemplo: Archivos `.mp3`.
 - **audio/wav:**
 - Archivos de audio en formato WAV.
 - Ejemplo: Archivos `.wav`.
 - **audio/ogg:**
 - Archivos de audio en formato OGG.
 - Ejemplo: Archivos `.ogg`.
 - **audio/aac:**
 - Archivos de audio en formato AAC.
 - Ejemplo: Archivos `.aac`.
-

5. Tipos de Video

Estos tipos representan formatos de video.

- **video/mp4:**
 - Videos en formato MP4.
 - Ejemplo: Archivos `.mp4`.
 - **video/webm:**
 - Videos en formato WebM.
 - Ejemplo: Archivos `.webm`.
 - **video/ogg:**
 - Videos en formato OGG.
 - Ejemplo: Archivos `.ogv`.
 - **video/x-msvideo:**
 - Videos en formato AVI.
 - Ejemplo: Archivos `.avi`.
-

6. Tipos de Fuentes

Estos tipos representan formatos de fuentes web.

- **font/woff:**
 - Fuentes en formato WOFF (Web Open Font Format).
 - Ejemplo: Archivos `.woff`.
 - **font/woff2:**
 - Fuentes en formato WOFF2 (versión optimizada).
 - Ejemplo: Archivos `.woff2`.
 - **font/ttf:**
 - Fuentes en formato TrueType.
 - Ejemplo: Archivos `.ttf`.
 - **font/otf:**
 - Fuentes en formato OpenType.
 - Ejemplo: Archivos `.otf`.
-

7. Tipos de Datos Relacionados con Modelos 3D

Estos tipos representan formatos de modelos 3D.

- **model/gltf+json:**
 - Modelos 3D en formato GLTF (JSON).
 - Ejemplo: Archivos `.gltf`.
 - **model/gltf-binary:**
 - Modelos 3D en formato GLTF binario.
 - Ejemplo: Archivos `.glb`.
-

8. Tipos Personalizados

Algunos tipos MIME pueden ser personalizados para necesidades específicas.

- **application/vnd.[nombre]:**
 - Tipos MIME personalizados para software específico.
 - Ejemplo: `application/vnd.ms-excel` para archivos de Excel.
-

Tipos MIME Más Usados en Desarrollo Web y APIs RESTful

Aunque existen muchos tipos MIME, en el desarrollo web y APIs RESTful, los siguientes son los más comunes:

1. **application/json:**
 - Respuestas de APIs RESTful.
2. **text/html:**
 - Páginas web renderizadas.
3. **text/css:**
 - Hojas de estilo.
4. **text/javascript:**
 - Código JavaScript.
5. **image/jpeg / image/png:**
 - Imágenes en aplicaciones web.
6. **application/pdf:**
 - Documentos descargables.
7. **application/xml:**
 - Datos en formato XML (usado en SOAP).

Conclusión

El protocolo HTTP es esencial para la comunicación en la web. Entender sus componentes (solicitudes, respuestas, encabezados, códigos de estado y tipos MIME) te permitirá desarrollar aplicaciones web eficientes y robustas. Además, este conocimiento es fundamental para trabajar con APIs RESTful y otras tecnologías modernas.

Los códigos de respuesta HTTP son fundamentales para la comunicación entre el cliente y el servidor. Entender su significado y uso correcto es crucial para desarrollar APIs RESTful eficientes y robustas. Los códigos más comunes en APIs RESTful incluyen **200 OK**, **201 Created**, **400 Bad Request**, **401 Unauthorized**, **404 Not Found** y **500 Internal Server Error**.

Los **tipos MIME** son esenciales para identificar correctamente el formato de los datos transferidos entre el cliente y el servidor. Aunque existen cientos de tipos MIME, en la práctica solo se utilizan unos pocos en el desarrollo web y APIs RESTful, como `application/json`, `text/html` y `image/jpeg`. Sin embargo, conocer la lista completa permite entender la diversidad de formatos disponibles y su propósito.